

Auditoría de Integración de Microservicios (G6 - Despacho)

Este documento registra el estado de integración de los demás grupos del ecosistema con los servicios del Grupo 6 (Despacho y Logística).

Grupo 4: Carrito y Checkout

* **URL del repositorio auditado:** `https://github.com/Mini-Marketplace-Cloud-UTEM/G4.git`

* **Fecha de revisión:** 05 de Julio de 2026

* **Estado de Integración:** ❌ **NO INTEGRADO**

Hallazgos de la Auditoría:

1. **Ignorando el costo de Despacho:** En su endpoint de `POST /v1/checkout` y en el cálculo total del carrito (`total\_amount`), **solamente** suman los precios de los productos consultados al Grupo 3 (Catálogo). No existe ninguna llamada al endpoint de G6 (`POST /api/v1/shipments/quotes`) para sumar el costo de envío.
2. **Fuera del mapa de integraciones:** En el `README.md` del Grupo 4, enumeran al Grupo 1 (Frontend), Grupo 2 (Auth), Grupo 3 (Catálogo), Grupo 5 (Pedidos), Grupo 7 (Inventario) y Grupo 8 (Pagos). El Grupo 6 no es mencionado en absoluto en su arquitectura de red.
3. **Falta de recolección de datos clave:** Para que G6 calcule un envío (Tarifa Consolidada), requiere el peso volumétrico y la ciudad de destino. Actualmente el Grupo 4 no recopila ni envía esta información durante el proceso de **Checkout**.

Acción Requerida (Mensaje para G4):

El equipo de G4 debe modificar su proceso de **Checkout** para:

1. Recibir la ciudad de destino y direcciones en el Payload inicial.
2. Realizar un request síncrono a G6: `POST https://g6-despacho-oficial.onrender.com/api/v1/shipments/quotes` pasando la ciudad y dimensiones.
3. Sumar la variable resultante (`total\_shipping\_cost`) al `total\_amount` de su carrito antes de pasarlo al Grupo 8 para ejecutar el pago real.

Ejemplo del payload esperado por G6 (a enviar por G4):

```
``json
{
  "orderId": "temp-cart-id",
  "city": "Santiago",
  "packages": [
    {
      "origin_cd": "CENTRO",
      "weight_kg": 2.5,
      "dimensions_cm": { "length": 20, "width": 20, "height": 20 }
    }
  ]
}
```

Grupo 1: Backend For Frontend (BFF)

* **URL del repositorio auditado:** `https://github.com/Mini-Marketplace-Cloud-UTEM/Grupo-1-BFF.git`

* **Fecha de revisión:** 05 de Julio de 2026

* **Estado de Integración:** ?? **INCOMPLETO / NO INICIADO**

Hallazgos de la Auditoría:

1. **Variable de entorno declarada pero no usada:** El README.md del Grupo 1 reconoce correctamente la existencia de G6 y lista la variable SHIPPING_SERVICE_URL apuntando al servicio de Despacho. Sin embargo, no hay ningún código que la utilice.
2. **Rutas ausentes:** No existe ningún router (outers/shipments.py) ni endpoints para consultar el estado de un despacho (GET /api/v1/shipments/{shipment_id}) ni su historial (GET /api/v1/shipments/{shipment_id}/history). De hecho, ni siquiera existen **stubs** (endpoints falsos) para el seguimiento de envíos.
3. **Fuera de su backlog inmediato:** En la sección **Próximos pasos** de su documentación, mencionan planes para implementar Catálogo, Carrito y Pedidos, pero omiten completamente la orquestación del seguimiento de Despachos.

Acción Requerida (Mensaje para G1):

El equipo de G1 debe extender su BFF para actuar como proxy (o gateway) de las consultas de estado de envío para que el Frontend (React) pueda consultarlo. Deben:

1. Crear un router para ****Despachos**** (ej. /v1/shipments).
2. Crear un endpoint GET /v1/shipments/{shipment_id} que redirija la llamada hacia GET https://g6-despacho-oficial.onrender.com/api/v1/shipments/{shipment_id}.
3. Crear un endpoint GET /v1/shipments/{shipment_id}/history que redirija la llamada hacia GET https://g6-despacho-oficial.onrender.com/api/v1/shipments/{shipment_id}/history.

Grupo 5: Pedidos

* ****URL del repositorio auditado:**** https://github.com/Mini-Marketplace-Cloud-UTEM/Grupo5-Pedidos.git
 * ****Fecha de revisión:**** 05 de Julio de 2026
 * ****Estado de Integración:**** ? ****ERROR DE CONTRATO (Integración Fallida)****

Hallazgos de la Auditoría:

1. ****Llamada incompleta:**** G5 está invocando el endpoint de Despacho POST /api/v1/shipments al momento de confirmar un pedido, pero ****solamente está enviando la lista de productos**** (packages).
2. ****Campos requeridos ausentes:**** G5 no está enviando order_id, customer_name, address ni city en el body del request, los cuales son campos estrictamente obligatorios en tu API (según ShipmentCreate). Esta llamada resultará siempre en un error HTTP 422 Unprocessable Entity.

Acción Requerida (Mensaje para G5):

El equipo de G5 debe ajustar el body de su request al invocar POST /api/v1/shipments. Deben inyectar el nombre del cliente, su dirección, la ciudad y el ID de la orden en el nivel superior del JSON, no solo el arreglo de packages.

Grupo 8: Pagos y Notificaciones

* ****URL del repositorio auditado:**** https://github.com/Mini-Marketplace-Cloud-UTEM/G8-Pagos-y-Notificaciones.git
 * ****Fecha de revisión:**** 05 de Julio de 2026
 * ****Estado de Integración:**** ? ****ERROR DE EVENTOS (Integración Fallida)****

Hallazgos de la Auditoría:

1. ****Discrepancia en el nombre del Evento:**** G8 está configurado para escuchar eventos asíncronos bajo el nombre literal "SHIPMENT_DELIVERED" (en mayúsculas/Screaming Snake Case). Sin embargo, tu servicio G6 emite el evento como "ShipmentDelivered" (en PascalCase). G8 rechazará todos tus eventos automáticamente por "UNSUPPORTED_EVENT_TYPE".

Acción Requerida (Mensaje para G8):

El equipo de G8 debe actualizar su lista de eventos soportados (SUPPORTED_EVENT_TYPES en su backend) y agregar "ShipmentDelivered" para que coincida exactamente con la capitalización del evento que emite Despacho.

Grupo 7: Reportes y Streaming

* ****URL del repositorio auditado:**** https://github.com/Mini-Marketplace-Cloud-UTEM/Grupo-7-Reportes-a-bash-y-Streaming.git
 * ****Fecha de revisión:**** 05 de Julio de 2026
 * ****Estado de Integración:**** ? ****ERROR DE PARSEO DE EVENTOS (Integración Fallida)****

Hallazgos de la Auditoría:

1. ****Payload Incompatible:**** G7 escucha correctamente el evento "ShipmentDelivered", pero su validador (Pydantic ShipmentDeliveredPayload) exige estrictamente que el evento contenga el campo delivered_at y exige que las llaves estén en snake_case (shipment_id, order_id). Tu evento de G6 no emite el campo delivered_at (utilizas occurredAt a nivel global y cambias estados internamente) y tu payload viaja en camelCase (shipmentId, orderId).

Acción Requerida (Mensaje para G7):

5/7/26, 9:14 p.m.

El equipo de G7 debe flexibilizar su modelo ShipmentDeliveredPayload. Deben:

1. Permitir leer shipmentId y orderId (camelCase) además de snake_case (o mapearlos usando un Alias de Pydantic).
2. Remover la exigencia estricta del campo delivered_at dentro del payload, o usar la propiedad global occurredAt de la envoltura del evento que t ya les proporciona.